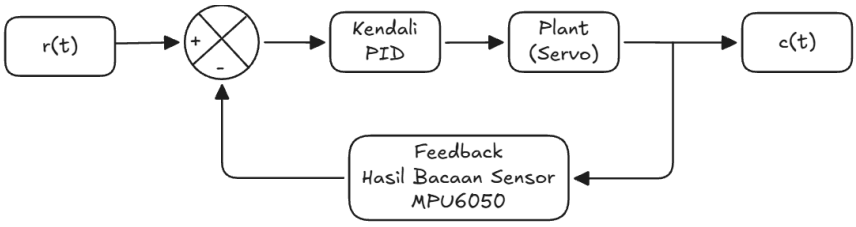
Proyek UTS
Studi Kasus Simulasi Kendali PID

A. Deskripsi Sistem

Pada proyek ini, kasus yang digunakan adalah desain gimbal single-axis sederhana yang menggunakan 3 (tiga) komponen utama, yaitu Arduino Nano, motor servo SG90, dan sensor IMU MPU6050 yang diintegrasikan dengan software Python. Software Python yang dikembangkan merupakan aplikasi desktop dengan library Tkinter yang dapat digunakan baik untuk simulasi (tanpa *hardware*), maupun pengujian langsung pada *hardware*. Tujuan dari sistem yang dikembangkan adalah menentukan tuning parameter PID yang optimal untuk mencapai target kendali sistem dengan error minimal, transien, dan stabil.

Mode Simulasi

Mode ini merupakan mode utama dari sistem yang dikembangkan, yaitu simulasi numerik tanpa perangkat fisik. Model sistem yang dikembangkan hanya berasal dari asumsi tanpa mempertimbangkan pengukuran hardware tertentu, misalnya identifikasi servo. Berikut gambaran arsitektur sistem feedback yang dikembangkan dalam simulasi ditunjukkan oleh Gambar 1.



Gambar 1. Blok Diagram Sistem Kendali

Pada simulasi ini, plant akan menghasilkan sudut virtual dengan noise Gaussian untuk mensimulasikan posisi servo. Dalam mode simulasi ini, terdapat beberapa visualisasi performa sistem yang dapat diamati, antara lain:

1. Sinyal step, memberikan sebuah sinyal (step) yang dimulai pada t=0,5s; metode ini dapat digunakan untuk mengamati karakteristik grafik respons tanggap

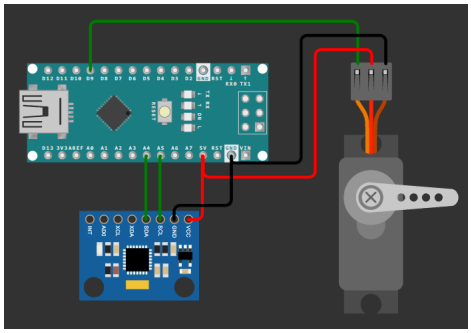
langkah yang berisi informasi rise time, settling time (keadaan tunak atau steady-state), overshoot, dan osilasi.

2. Sinyal impulse, memberikan pulsa berdurasi 0,1s pada t=0,5s; metode ini dapat digunakan untuk mengamati karakteristik transien sistem, yaitu mensimulasikan gangguan yang tiba-tiba.
3. Sinyal sinus, memiliki persamaan matematis $A \sin(2\pi \times 0.5 \times (t-0.5))$ yang disimulasikan dengan frekuensi 0,5Hz yang dimulai pada t=0,5s.
4. Target manual, merupakan pemberian target posisi secara manual oleh pengguna untuk melihat respons sistem kendali dari servo.

Mode Hardware (Arduino)

Seperti pada penjelasan sebelumnya, *hardware* terdiri atas 3 (tiga) komponen, yaitu Arduino Nano, motor servo, dan sensor MPU6050. Fungsi hardware yang dijalankan adalah menggantikan simulasi servo dari Mode Simulasi, dengan melakukan ekstraksi data sensor menjadi sudut Euler (roll, pitch, yaw) dan menerima perintah gerak (posisi akhir sudut) servo dari software. Secara sederhana, prinsip dari mode ini adalah melakukan kalkulasi PID pada software, kemudian melalui komunikasi serial meminta *hardware* untuk eksekusi (menggerakkan servo, membaca sensor, dan sebagainya).

Dikarenakan mode ini memerlukan hardware untuk pengujian langsung, berikut akan dibahas terlebih dahulu persiapan rangkaian dan firmware untuk sistem. Rangkaian utama tersusun atas 3 (tiga) komponen yang telah disebutkan sebelumnya dengan skematik seperti ditunjukkan pada Gambar 2.

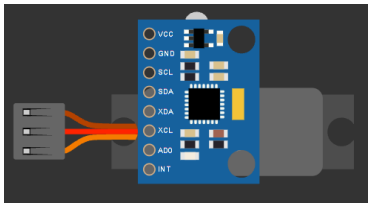


Gambar 2. Skematik Rangkaian

Berdasarkan susunan rangkaian tersebut, servo dan sensor terhubung ke Arduino secara langsung dengan catu daya terintegrasi. Pin sinyal PWM servo dihubungkan ke pin D9

Arduino, sedangkan pin SDA dan SCL sensor masing-masing ke pin A4 dan A5 menyesuaikan dengan datasheet pinout yang ada.

Jika dikaji dari struktur mekanisnya, sensor MPU6050 dipasang pada lengan servo atau bagian yang diputar oleh servo. Hal ini dilakukan karena sensor digunakan sebagai umpan balik posisi servo. Dengan mengetahui keadaan aktual posisi servo, maka sistem kendali dapat mengkalkulasi error-nya untuk menuju target yang diinginkan. Visualisasi pemasangan mekanis ditunjukkan pada Gambar 2.



Gambar 2. Pemasangan MPU6050 pada Servo

Dengan kondisi tersebut, servo diatur pada sudut netral 90 derajat yang relatif terhadap sudut 0 derajat pada MPU6050. Titik ini dianggap sebagai titik inisialisasi, sekaligus titik yang akan dijadikan referensi atau target sistem kendali. Sistem kendali akan mempertahankan posisi 0 derajat sensor dengan menggerakkan servo sesuai umpan balik negatifnya (berdasarkan error). Konsep ini merupakan implementasi sederhana gimbal satu sumbu (single-axis) di mana sistem mempertahankan posisi meskipun terdapat gangguan atau perubahan sudut eksternal pada badan dudukan.

B. Hasil dan Analisis

Hasil pengujian diperoleh dari 3 (tiga) skenario, yaitu simulasi step, simulasi impuls, dan implementasi *hardware*. Untuk mencapai target yang diharapkan dengan karakteristik cepat dan stabil, dilakukan tuning berbasis Ziegler-Nichols metode 1 dan 2 terlebih dahulu. Nilai ini akan dijadikan nilai parameter awal untuk fine tuning dengan metode empiris (trial and error). Untuk pengujian ini, dibuat skrip Python dan data CSV terpisah yang mensimulasikan parameter tertentu dan perubahannya agar lebih cepat dalam proses pengujian dengan dasar aplikasi yang telah dikembangkan (kode tidak disertakan dalam *repository* karena bukan merupakan kode utama).

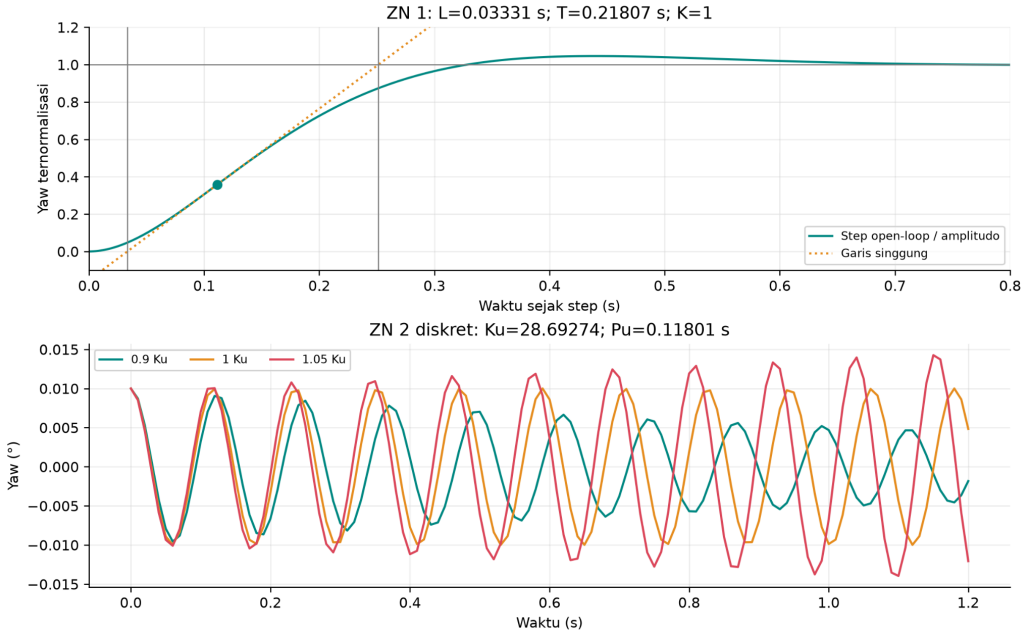
Berikut rancangan eksperimen yang dilakukan ditunjukkan pada Tabel 1.

Tabel 1. Rancangan Eksperimen PID

Parameter	Nilai
Plant	$G(s) = \frac{100}{s^2+14s+100}$, dengan ω_n=10rad/s; ζ=0,7
Sinyal Step	0° ke 20° pada t=0,5s
Sinyal Impulse	20° untuk 0,1s dimulai dari t=0,5s; kemudian kembali ke 0°

ZN 1st dan 2ndMethod

1. Identifikasi ZN (ditunjukkan pada Gambar 3)



Gambar 3. Grafik Step Response untuk Mencari Nilai L, T, K_{cr}, dan P_{cr}

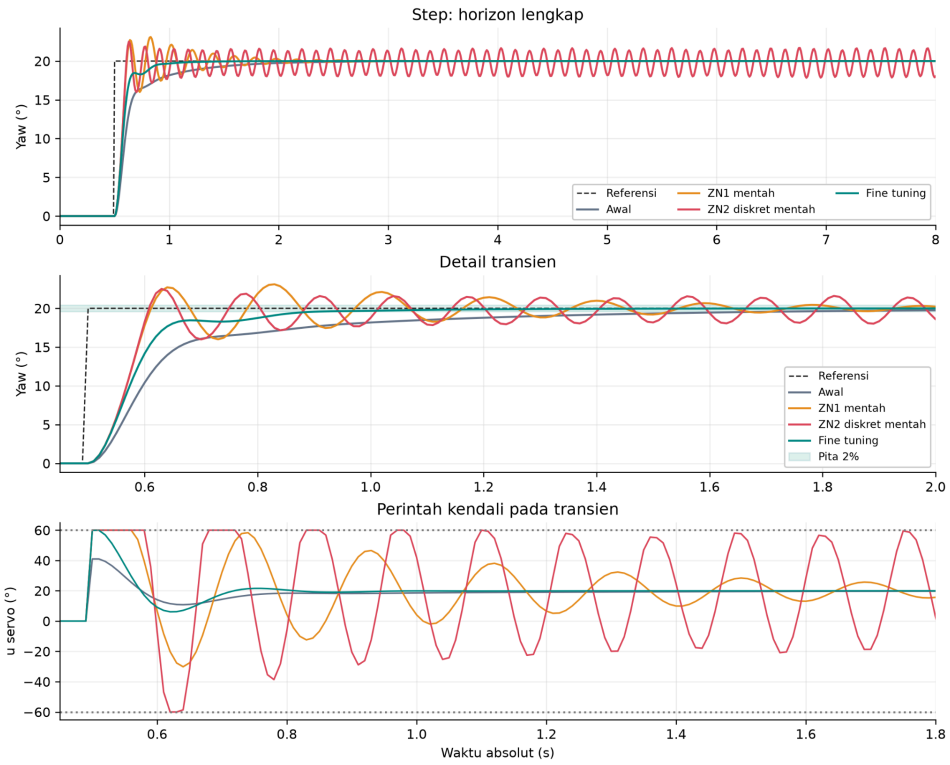
2. Dari grafik tersebut diperoleh nilai L=0,03 dan T=0,21 untuk ZN metode 1, serta K_{cr}=28,69 dan P_{cr}=0,12s untuk ZN metode 2. Dengan demikian, untuk kendali PID, berdasarkan tabel ZN dipeorleh hasil seperti ditunjukkan pada Tabel 2.

Tabel 2. Nilai Parameter Berdasarkan Kalkulasi Tabel ZN

PID	Kp	Ti	Ki	Td	Kd
ZN 1	1.2T/L = 7,86	2L = 0,06	Kp(1/Ti) = 117,94	L/2 = 0,02	Kp.Td = 0,13
ZN 2	$0.6K_{cr}$ = 17,22	$0.5P_{cr}$ = 0,06	Kp(1/Ti) = 297,77	$0.125K_{cr}$ = 0,01	Kp.Td = 0,25

3. Nilai-nilai tersebut diimplementasikan ke dalam sistem (aplikasi) dan memperoleh hasil step response pada Gambar 4 dan impulse response pada Gambar 5.

   Step Response



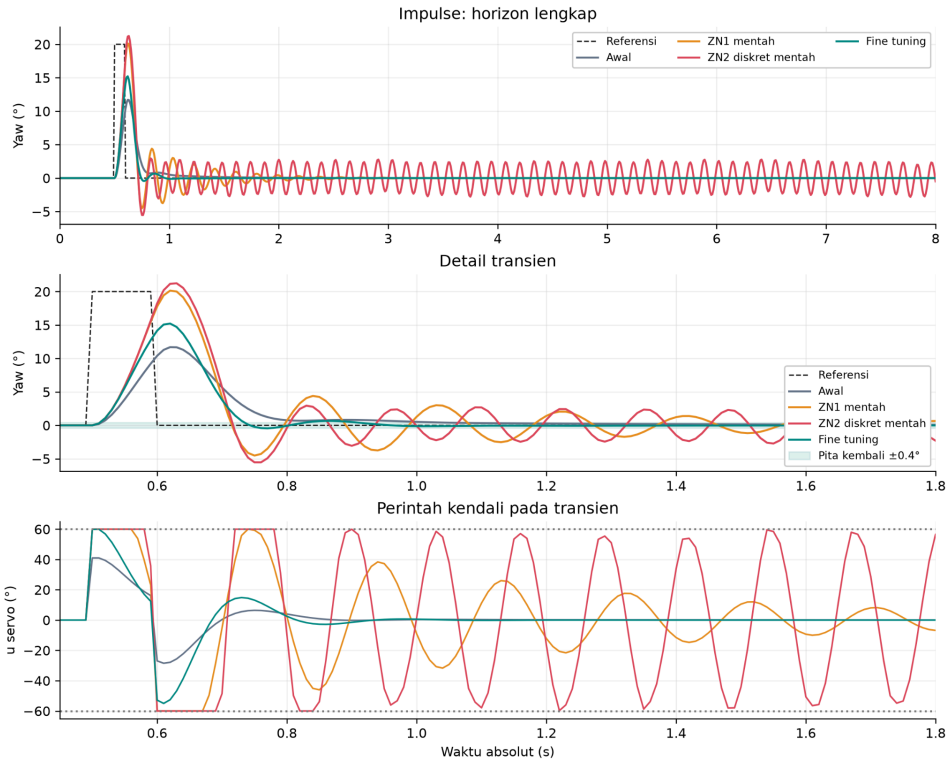
Gambar 4. Grafik Step Response Target, ZN 1, ZN 2, dan Fine Tuning

Didapatkan tabel perbandingan karakteristik sistem yang ditunjukkan pada Tabel 3 berikut.

Tabel 3. Perbandingan Karakteristik Sistem

Tuning	Rise Time (s)	Overshoot (%)	Settling Time (s)	IAE (°.s)
ZN 1	0.08	15.51	1.3	2.87
ZN 2	0.07	12.57	-	9.91
Fine Tuning	0.12	0	0.42	2.09

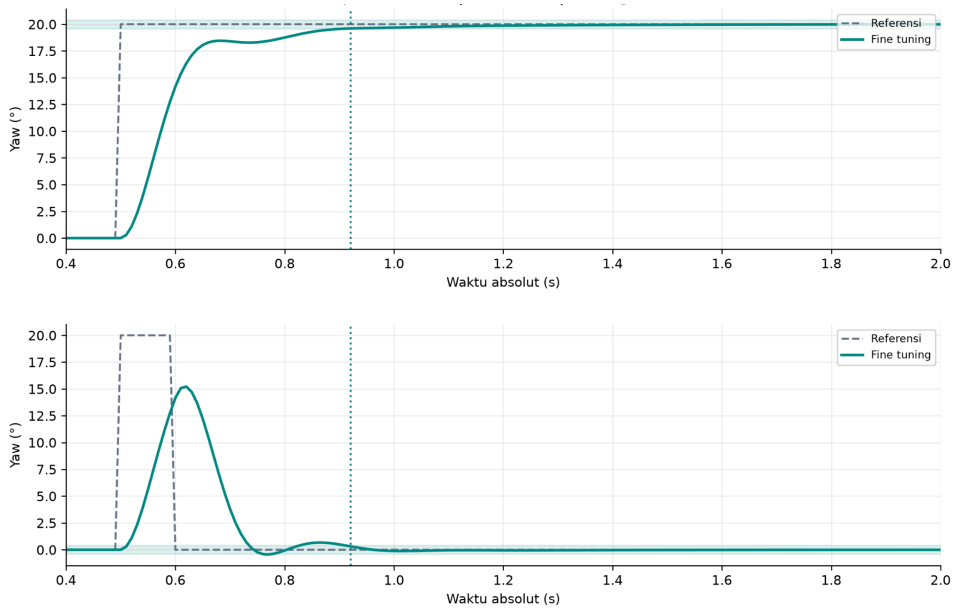
Impulse Response



Gambar 5. Grafik Impulse Response Target, ZN 1, ZN 2, dan Fine Tuning

4. Fine Tuning:

   Sebelumnya, masing-masing hasil metode ZN telah dilakukan *fine-tuning* dengan metode *trial and error*, untuk memperjelas hasil dapat diamati pada Gambar 6.



Gambar 6. Hasil Fine Tuning Sistem Kendali PID

Implementasi ke Hardware

Nilai hasil fine tuning dari simulasi diuji secara langsung pada hardware, yaitu:

- Kp = 2,99
- Ki = 10,54
- Kd = 0,15

Ternyata apabila diimplementasikan pada hardware, nilai tersebut belum optimal, dan diperoleh nilai optimal untuk hardware langsung, yaitu:

- Kp = 2
- Ki = 5,12
- Kd = 0,12

Hal ini menunjukkan bahwa simulasi memang tidak dapat merepresentasikan keadaan nyata karena pemodelan yang dilakukan juga tidak melalui identifikasi hardware terlebih dahulu, melainkan melalui asumsi/perkiraan.

C. Kesimpulan dan Saran

Metode ini mungkin tidak terlalu praktis dan efektif dalam implementasi teknis, namun cukup mudah menggambarkan atau memvisualisasikan hasil dari simulasi PID dengan baik. Kendala utama yang dihadapi adalah *drifting* sensor IMU terutama sudut yaw yang digunakan sebagai sumbu utama kendali, namun sistem kendali cukup stabil dalam

mengatasi gangguan tersebut. Saran untuk penelitian selanjutnya adalah menggunakan sensor IMU absolut seperti BNO055 atau penggunaan servo dengan sensor pembacaan sudut (misalnya *rotary encoder*) untuk meningkatkan akurasi posisi akhir atau hasil target. Selain itu, diperlukan identifikasi hardware yang akurat untuk memastikan simulasi berjalan sesuai dengan hardware langsung.

Catatan:

Kode program dapat dilihat pada link GitHub berikut.
(https://github.com/therafaeliger/pmsk-pid-project.git)